

JOINTURES SOUS SQL

Suite(4)

REQUETE MULTI-TABLES

- La requête multi-tables permet de confronter des colonnes dans des tables différentes mais liées à partir de la clé étrangère ou autres colonnes communes.

Syntaxe:

```
Select table1.colonne1,table2.colonne1,table2.colonne2 from table 1,table 2  
where table1.id_tab1=table2.id_tab1
```

Exemple:

```
select client.nom,facture.montant from client,facture where client.ncli=facture.ncli ;
```

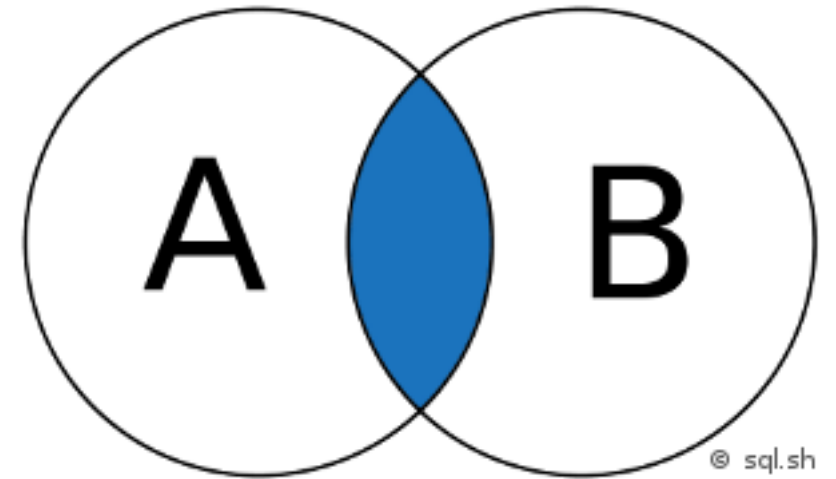


INNER JOIN

- C'est une jointure interne pour retourner les enregistrements quand la condition est vraie dans les 2 tables.

Syntaxe:

```
SELECT * FROM A INNER JOIN B ON A.key =  
B.key
```



INNER JOIN

```
SELECT id, prenom, nom, date_achat,  
num_facture, prix_total FROM utilisateur  
INNER JOIN commande ON utilisateur.id =  
commande.utilisateur_id
```

Exemple:

Table utilisateur :

id	prenom	nom	email	ville
1	Aimée	Marechal	aime.marechal@example.com	Paris
2	Esmée	Lefort	esmee.lefort@example.com	Lyon
3	Marine	Prevost	m.prevost@example.com	Lille
4	Luc	Rolland	lucrolland@example.com	Marseille

Table commande :

utilisateur_id	date_achat	num_facture	prix_total
1	2013-01-23	A00103	203.14
1	2013-02-14	A00104	124.00
2	2013-02-17	A00105	149.45
2	2013-02-21	A00106	235.35
5	2013-03-02	A00107	47.58

Résultats :

id	prenom	nom	date_achat	num_facture	prix_total
1	Aimée	Marechal	2013-01-23	A00103	203.14
1	Aimée	Marechal	2013-02-14	A00104	124.00
2	Esmée	Lefort	2013-02-17	A00105	149.45
2	Esmée	Lefort	2013-02-21	A00106	235.35

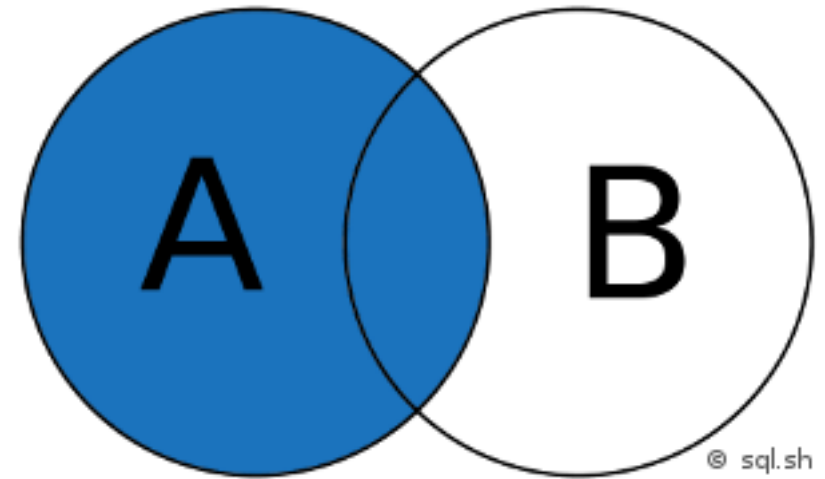


LEFT JOIN

- Jointure externe pour retourner tous les enregistrements de la table gauche (LEFT = gauche) même si la condition n'est pas vérifiée dans l'autre table.

Syntaxe:

```
SELECT * FROM A LEFT JOIN B ON A.key =  
B.key
```



LEFT JOIN

Table utilisateur :

id	prenom	nom	email	ville
1	Aimée	Marechal	aime.marechal@example.com	Paris
2	Esmée	Lefort	esmee.lefort@example.com	Lyon
3	Marine	Prevost	m.prevost@example.com	Lille
4	Luc	Rolland	lucrolland@example.com	Marseille

Résultats :

id	prenom	nom	date_achat	num_facture	prix_total
1	Aimée	Marechal	2013-01-23	A00103	203.14
1	Aimée	Marechal	2013-02-14	A00104	124.00
2	Esmée	Lefort	2013-02-17	A00105	149.45
2	Esmée	Lefort	2013-02-21	A00106	235.35
3	Marine	Prevost	NULL	NULL	NULL
4	Luc	Rolland	NULL	NULL	NULL

Table commande :

utilisateur_id	date_achat	num_facture	prix_total
1	2013-01-23	A00103	203.14
1	2013-02-14	A00104	124.00
2	2013-02-17	A00105	149.45
2	2013-02-21	A00106	235.35
5	2013-03-02	A00107	47.58

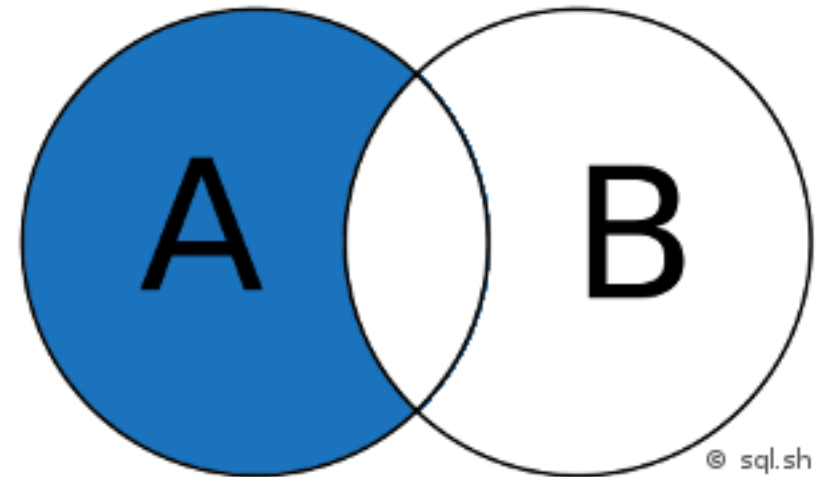
```
SELECT * FROM utilisateur LEFT  
JOIN commande ON utilisateur.id  
= commande.utilisateur_id
```

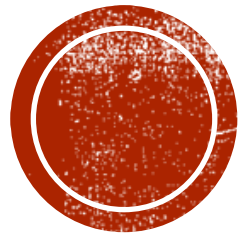
LEFT JOIN(SANS INTERSECTION)

- Jointure externe pour retourner tous les enregistrements de la table gauche (LEFT = gauche) même si la condition n'est pas vérifiée dans l'autre table.

Syntaxe:

```
SELECT * FROM A LEFT JOIN B ON A.key = B.key  
WHERE B.key IS NULL
```





SQL SOUS-REQUÊTE

SQL SOUS-REQUÊTE

- Dans le langage SQL une sous-requête (aussi appelée “requête imbriquée” ou “requête en cascade”) consiste à exécuter une requête à l’intérieur d’une autre requête. Une requête imbriquée est souvent utilisée au sein d’une clause WHERE ou de HAVING pour remplacer une ou plusieurs constantes.

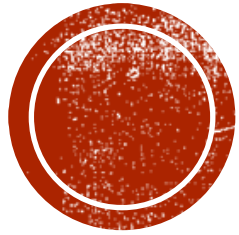
Syntaxe:

```
SELECT * FROM table WHERE nom_colonne = ( SELECT 'valeur' FROM table2  
LIMIT 1 )
```

Exemples:

- Select nom,adresse from client where ncli=(select ncli from facture where montant>10000 limit 1)
- Select nom,adresse from client where ncli in(select ncli from facture where montant>10000)





VUE EN SQL

CREATION DE VUE EN SQL

- Une vue dans une base de données est une synthèse d'une requête d'interrogation de la base. On peut la voir comme une table virtuelle, définie par une requête.

Syntaxe:

```
CREATE VIEW <NOM DE VUE> [ (LISTE D'ATTRIBUT)]  
AS <QUESTION>  
[WITH CHECK OPTION]
```

Exemple:

```
Create view infoclient(nom,adresse) as select nom,adresse from client;
```

```
Select*f rom infoclient;
```



SUPPRESSION DE VUE EN SQL

- La suppression d'une vue s'effectue simplement par la commande :

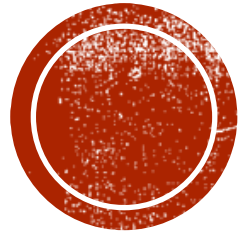
Syntaxe:

DROP VIEW <NOM DE VUE>.

Exemple:

Drop view infoclient;





UTILISATEURS ET LES PRIVILÈGES

CRÉATION D'UTILISATEUR

Syntaxe:

- Pour créer un utilisateur facilement, on peut utiliser la syntaxe suivante:

```
CREATE USER 'user'@'localhost' IDENTIFIED BY 'password';
```

Exemple:

```
Create user 'rakoto'@'localhost' identified by '1234';
```



ATTRIBUTION DE PRIVILÈGES

Syntaxe:

- On peut attribuer des privilèges à un utilisateur à l'aide du mot clé GRANT:

```
GRANT SELECT, INSERT, UPDATE, DELETE ON 'database'.* TO 'user'@'localhost';
```

Exemple:

```
GRANT SELECT, INSERT, UPDATE ON 'gestionclient'.* TO 'rakoto'@'localhost';
```

ou

```
GRANT ALL ON 'gestionclient'.* TO 'rakoto'@'localhost';
```



SUPPRESSION DE PRIVILÈGES ET UTILISATEUR

Syntaxe:

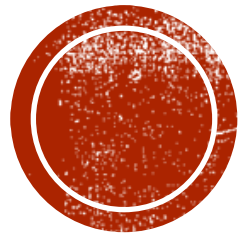
- Pour révoquer l'ensemble des droits d'un utilisateur, on peut lancer la requête suivante.
 - `REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'user'@'localhost';`
 - `REVOKE DELETE ON database.* FROM 'user'@'localhost';`

`DROP USER 'user'@'localhost';`

Exemple:

- `REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'rakoto'@'localhost';`
- `REVOKE DELETE ON gestionclient* FROM 'rakoto'@'localhost';`





PROCÉDURE STOCKÉE



PROCÉDURE STOCKÉE

Une **procédure stockée**, aussi appelée **stored procedure** en anglais, est un concept utilisé en administration de base de données afin d'exécuter un ensemble d'instructions SQL.

Exemple:

```
DELIMITER //  
CREATE PROCEDURE nomclient  
(IN con CHAR(20))  
BEGIN  
    SELECT nom  
    FROM client  
    WHERE cat=con;  
END //  
DELIMITER ;
```

Pour appeler cette procédure, on peut écrire: `CALL nomclient('2');`

